



NATIONAL UNIVERSITY OF TECHNOLOGY, ISLAMABAD

Database Systems PBL PROJECT REPORT

Project Title:

University Enrollment Management Systems

Submitted to: Ma'am Saima Yasmeen

Submitted by:
Bachelor of Software Engineering
Batch: 25

SR.NO.	Name	Student ID
1	Sana Ahmad	F25608001
2	Mehmoona Akbar	F25608007
3	Ishmal Malik	F25608011
4	Sawaira Nadeem	F25608020
5	Laiba Idrees	F25608042

Date of Submission: 11-June-2026

Table of Contents

1. Introduction
2. Project Objectives
3. System Overview
4. Business Rules
5. Enhanced Entity Relationship Diagram (EERD)
6. Relational Schema Design
7. SQL Implementation
8. Sample Data Population
9. Views
10. Trigger
11. SQL Queries and Results
12. Conclusion
13. GitHub Repository Link

1. Introduction

The University Enrollment Management System is a database application designed to manage students, instructors, departments, courses, sections, enrollments, and payment records efficiently. The system provides a centralized database structure that supports academic and administrative operations while maintaining data integrity through relational constraints.

2. Project Objectives

The objectives of this project are:

- Design a complete database using EERD.
- Implement relational tables in SQL Server.
- Enforce primary key and foreign key constraints.
- Maintain data integrity using CHECK, UNIQUE, and NOT NULL constraints.
- Create views and triggers.
- Perform complex SQL queries including joins, aggregation, and subqueries.

3. System Overview

The system manages:

- Student records
- Department information
- Instructor details

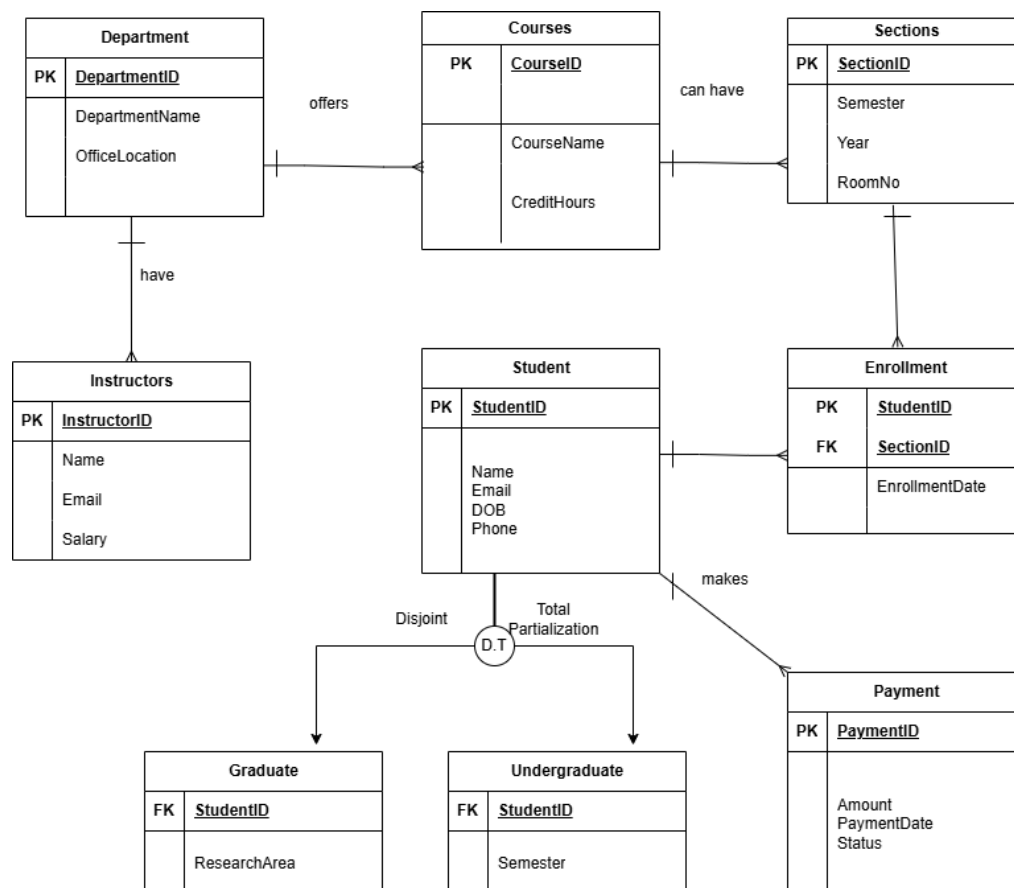
- Course offerings
- Section management
- Student enrollments
- Fee payments
- Audit tracking

The database ensures accurate and consistent storage of academic information.

4. Business Rules

1. A department can offer multiple courses.
2. A department can have multiple instructors.
3. Each course belongs to one department.
4. A course can have multiple sections.
5. A student can enroll in multiple sections.
6. Students can make multiple payments.
7. Every payment is associated with a student.
8. Undergraduate and Graduate students are specializations of Student.

5. Enhanced Entity Relationship Diagram (EERD)



The EERD represents all entities, attributes, relationships, cardinalities, and specialization structures used in the University Enrollment Management System.

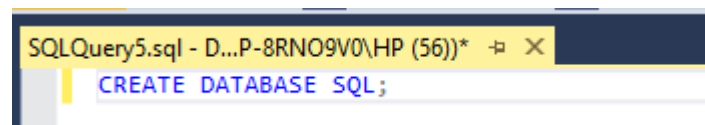
6. Relational Schema Design

Tables:

- Department
- Student
- UndergraduateStudent
- GraduateStudent
- Instructor
- Course
- Section
- Enrollment
- Payment
- AuditLog

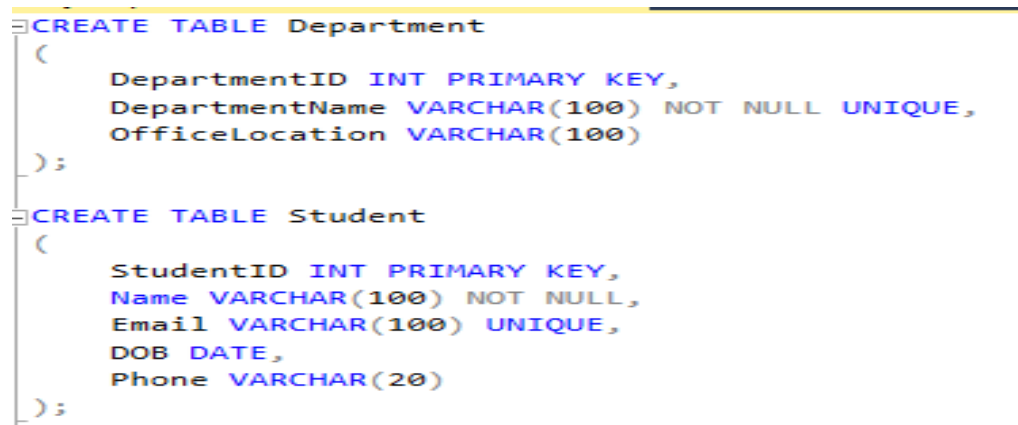
7. SQL Implementation

Create Database:



CREATE TABLE:

- Department/Student



- Instructor

```
CREATE TABLE Instructor
(
    InstructorID INT PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Email VARCHAR(100) NOT NULL UNIQUE,
    Salary DECIMAL(10,2) CHECK (Salary > 0),
    DepartmentID INT NULL,

    CONSTRAINT FK_Instructor_Department
    FOREIGN KEY (DepartmentID)
    REFERENCES Department(DepartmentID)
    ON DELETE SET NULL
);
```

- Graduate/Undergraduate Student

```
CREATE TABLE UndergraduateStudent
(
    StudentID INT PRIMARY KEY,
    Semester INT CHECK (Semester BETWEEN 1 AND 8),

    CONSTRAINT FK_Undergraduate_Student
    FOREIGN KEY (StudentID)
    REFERENCES Student(StudentID)
);

CREATE TABLE GraduateStudent
(
    StudentID INT PRIMARY KEY,
    ResearchArea VARCHAR(100),

    CONSTRAINT FK_Graduate_Student
    FOREIGN KEY (StudentID)
    REFERENCES Student(StudentID)
);
```

- Course

```
CREATE TABLE Course
(
    CourseID INT PRIMARY KEY,
    CourseName VARCHAR(100) NOT NULL,
    CreditHours INT CHECK (CreditHours BETWEEN 1 AND 6),
    DepartmentID INT NOT NULL,

    CONSTRAINT FK_Course_Department
    FOREIGN KEY (DepartmentID)
    REFERENCES Department(DepartmentID)
    ON DELETE CASCADE
);
```

- Section

```
CREATE TABLE Section
(
    SectionID INT PRIMARY KEY,
    Semester VARCHAR(20),
    YearValue INT,
    RoomNo VARCHAR(20),

    CourseID INT NOT NULL,
    InstructorID INT NULL,

    CONSTRAINT FK_Section_Course
    FOREIGN KEY (CourseID)
    REFERENCES Course(CourseID)
    ON DELETE CASCADE,

    CONSTRAINT FK_Section_Instructor
    FOREIGN KEY (InstructorID)
    REFERENCES Instructor(InstructorID)
    ON DELETE SET NULL
);
```

- Enrollment

```
CREATE TABLE Enrollment
(
    EnrollmentID INT PRIMARY KEY,

    StudentID INT NOT NULL,
    SectionID INT NOT NULL,

    EnrollmentDate DATE,
    Grade CHAR(2),

    CONSTRAINT FK_Enrollment_Student
    FOREIGN KEY (StudentID)
    REFERENCES Student(StudentID),

    CONSTRAINT FK_Enrollment_Section
    FOREIGN KEY (SectionID)
    REFERENCES Section(SectionID)
);
```

- Payment/Auditlog

```
CREATE TABLE Payment
(
    PaymentID INT PRIMARY KEY,

    StudentID INT NOT NULL,

    Amount DECIMAL(10,2)
    CHECK (Amount > 0),

    PaymentDate DATE,

    Status VARCHAR(20)
    CHECK (Status IN ('Paid', 'Pending', 'Cancelled')),

    CONSTRAINT FK_Payment_Student
    FOREIGN KEY (StudentID)
    REFERENCES Student(StudentID)
);

CREATE TABLE AuditLog
(
    LogID INT IDENTITY(1,1) PRIMARY KEY,
    StudentID INT,
    ActionType VARCHAR(100),
    ActionDate DATETIME DEFAULT GETDATE()
);
```

The database was implemented using Microsoft SQL Server.

Constraints implemented:

- PRIMARY KEY
- FOREIGN KEY
- NOT NULL
- UNIQUE
- CHECK

8. Sample Data Population

INSERT INTO DEPARTMENT

```
INSERT INTO Department VALUES  
(1, 'Computer Science', 'Block A'),  
(2, 'Software Engineering', 'Block B'),  
(3, 'Information Technology', 'Block C'),  
(4, 'Electrical Engineering', 'Block D'),  
(5, 'Business Administration', 'Block E');
```

INSERT INTO STUDENTS

```
INSERT INTO Student VALUES  
(101, 'Ali Khan', 'ali101@gmail.com', '2003-01-15', '03001234567'),  
(102, 'Ahmed Raza', 'ahmed102@gmail.com', '2002-05-10', '03011234567'),  
(103, 'Sara Ahmed', 'sara103@gmail.com', '2003-07-12', '03021234567'),  
(104, 'Ayesha Noor', 'ayesha104@gmail.com', '2004-02-18', '03031234567'),  
(105, 'Usman Tariq', 'usman105@gmail.com', '2002-11-11', '03041234567'),  
(106, 'Fatima Ali', 'fatima106@gmail.com', '2003-03-15', '03051234567'),  
(107, 'Bilal Khan', 'bilal107@gmail.com', '2003-08-25', '03061234567'),  
(108, 'Hina Malik', 'hina108@gmail.com', '2002-09-14', '03071234567'),  
(109, 'Hamza Shah', 'hamza109@gmail.com', '2003-10-19', '03081234567'),  
(110, 'Zara Iqbal', 'zara110@gmail.com', '2004-01-01', '03091234567'),  
(111, 'Daniyal', 'daniyal111@gmail.com', '2003-06-22', '03101234567'),  
(112, 'Maryam', 'maryam112@gmail.com', '2002-12-30', '03111234567'),  
(113, 'Asad', 'asad113@gmail.com', '2003-04-17', '03121234567'),  
(114, 'Nida', 'nida114@gmail.com', '2002-02-11', '03131234567'),  
(115, 'Farhan', 'farhan115@gmail.com', '2004-08-20', '03141234567');
```

INSERT INTO UN_GRADUATE STD

```
INSERT INTO UndergraduateStudent VALUES
(101,3),
(102,5),
(103,4),
(104,2),
(105,6),
(106,3),
(107,7),
(108,5);
```

INSERT INTO GRADUATE STD

```
INSERT INTO GraduateStudent VALUES
(109,'Artificial Intelligence'),
(110,'Cyber Security'),
(111,'Data Science'),
(112,'Machine Learning'),
(113,'Cloud Computing'),
(114,'Blockchain'),
(115,'Software Testing');
```

INSERT INTO INSTRUCTOR

```
INSERT INTO Instructor VALUES
(1,'Dr. Hassan','hassan@uni.edu',120000,1),
(2,'Dr. Ahmed','ahmed@uni.edu',125000,1),
(3,'Dr. Sara','sara@uni.edu',118000,2),
(4,'Dr. Bilal','bilal@uni.edu',130000,3),
(5,'Dr. Ayesha','ayesha@uni.edu',122000,4);
```

INSERT INTO COURSE

```
INSERT INTO Course VALUES
(1,'Database Systems',3,1),
(2,'Data Structures',3,1),
(3,'Operating Systems',3,1),
(4,'Software Engineering',3,2),
(5,'Web Development',3,2),
(6,'Networking',3,3),
(7,'Digital Logic',3,4),
(8,'Marketing',3,5);
```


INSERT INTO SECTION

```
INSERT INTO Section VALUES
(1, 'Spring', 2026, 'A101', 1, 1),
(2, 'Spring', 2026, 'A102', 2, 2),
(3, 'Spring', 2026, 'B201', 3, 1),
(4, 'Spring', 2026, 'B202', 4, 3),
(5, 'Spring', 2026, 'C301', 5, 4),
(6, 'Spring', 2026, 'C302', 6, 5);
```

INSERT INTO ENTROLLMENT

```
INSERT INTO Enrollment VALUES
(1, 101, 1, '2026-01-10', 'A'),
(2, 102, 1, '2026-01-10', 'B'),
(3, 103, 2, '2026-01-10', 'A'),
(4, 104, 2, '2026-01-10', 'B'),
(5, 105, 3, '2026-01-10', 'A'),
(6, 106, 3, '2026-01-10', 'B'),
(7, 107, 4, '2026-01-10', 'A'),
(8, 108, 4, '2026-01-10', 'C'),
(9, 109, 5, '2026-01-10', 'A'),
(10, 110, 5, '2026-01-10', 'B'),
(11, 111, 6, '2026-01-10', 'A'),
(12, 112, 6, '2026-01-10', 'B');
```

INSERT INTO PAYMENT

```
INSERT INTO Payment VALUES
(1, 101, 50000, '2026-01-15', 'Paid'),
(2, 102, 55000, '2026-01-15', 'Paid'),
(3, 103, 45000, '2026-01-15', 'Pending'),
(4, 104, 60000, '2026-01-15', 'Paid'),
(5, 105, 40000, '2026-01-15', 'Pending'),
(6, 106, 65000, '2026-01-15', 'Paid'),
(7, 107, 50000, '2026-01-15', 'Paid'),
(8, 108, 35000, '2026-01-15', 'Cancelled'),
(9, 109, 70000, '2026-01-15', 'Paid'),
(10, 110, 55000, '2026-01-15', 'Paid');
```

The database was populated with realistic sample records to test relationships and query execution.

9. Views

View 1: StudentCourseView

```
CREATE VIEW StudentCourseView AS
SELECT
  s.StudentID,
  s.Name,
  c.CourseName,
  sec.Semester
FROM Student s
JOIN Enrollment e
  ON s.StudentID=e.StudentID
JOIN Section sec
  ON e.SectionID=sec.SectionID
JOIN Course c
  ON sec.CourseID=c.CourseID;
```

Purpose:

Displays student enrollment information along with course details.

View 2: DepartmentInstructorView

```
CREATE VIEW DepartmentInstructorView AS
SELECT
  d.DepartmentName,
  i.Name,
  i.Email
FROM Department d
JOIN Instructor i
  ON d.DepartmentID=i.DepartmentID;
```

Purpose:

Displays instructors assigned to each department.

10. Trigger

Trigger Name: trg_payment_audit

```
CREATE TRIGGER trg_payment_audit
ON Payment
AFTER INSERT
AS
BEGIN
  INSERT INTO AuditLog(StudentID, ActionType)
  SELECT StudentID, 'Payment Added'
  FROM inserted;
END;
```

Purpose:

Automatically records payment activity in the AuditLog table whenever a new payment record is inserted.

11. SQL Queries and Results

Multi-Table Join Query 1

```
SELECT
s.Name AS StudentName,
c.CourseName,
i.Name AS InstructorName
FROM Student s
JOIN Enrollment e
ON s.StudentID=e.StudentID
JOIN Section sec
ON e.SectionID=sec.SectionID
JOIN Course c
ON sec.CourseID=c.CourseID
JOIN Instructor i
ON sec.InstructorID=i.InstructorID;
```

	StudentName	CourseName	InstructorName
1	Ali Khan	Database Systems	Dr. Hassan
2	Ahmed Raza	Database Systems	Dr. Hassan
3	Sara Ahmed	Data Structures	Dr. Ahmed
4	Ayesha Noor	Data Structures	Dr. Ahmed
5	Usman Tariq	Operating Systems	Dr. Hassan
6	Fatima Ali	Operating Systems	Dr. Hassan
7	Bilal Khan	Software Engine...	Dr. Sara
8	Hina Malik	Software Engine...	Dr. Sara
9	Hamza Shah	Web Development	Dr. Bilal
10	Zara Iqbal	Web Development	Dr. Bilal
11	Daniyal	Networking	Dr. Ayesha
12	Maryam	Networking	Dr. Ayesha

Explanation:

Retrieves student, course, and instructor information.

Multi-Table Join Query 2

```
SELECT
    d.DepartmentName,
    c.CourseName,
    i.Name AS InstructorName
FROM Department d
JOIN Course c
ON d.DepartmentID=c.DepartmentID
JOIN Section sec
ON c.CourseID=sec.CourseID
JOIN Instructor i
ON sec.InstructorID=i.InstructorID;
```

102 %

Results Messages

	DepartmentName	CourseName	InstructorName
1	Computer Science	Database Systems	Dr. Hassan
2	Computer Science	Data Structures	Dr. Ahmed
3	Computer Science	Operating Systems	Dr. Hassan
4	Software Engineering	Software Engineering	Dr. Sara
5	Software Engineering	Web Development	Dr. Bilal
6	Information Technology	Networking	Dr. Ayesha

Explanation:

Displays department, course, and instructor details.

Aggregation Query 1

```
SELECT
    Status,
    COUNT(*) AS TotalPayments,
    SUM(Amount) AS TotalRevenue
FROM Payment
GROUP BY Status;
```

Results Messages

	Status	TotalPayments	TotalRevenue
1	Cancelled	1	35000.00
2	Paid	7	405000.00
3	Pending	2	85000.00

Explanation:

Shows payment statistics grouped by payment status.

Aggregation Query 2

```
SELECT
StudentID,
AVG(Amount) AS AverageFee
FROM Payment
GROUP BY StudentID
HAVING AVG(Amount) > 50000;
```

Results		Messages	
	StudentID	AverageFee	
1	102	55000.000000	
2	104	60000.000000	
3	106	65000.000000	
4	109	70000.000000	
5	110	55000.000000	

Explanation:

Calculates average payment amounts per student.

Nested Subquery 1

```
SELECT Name
FROM Student
WHERE StudentID IN
(
SELECT StudentID
FROM Payment
WHERE Amount > 50000
);
```

Results		Messages	
	Name		
1	Ahmed Raza		
2	Ayesha Noor		
3	Fatima Ali		
4	Hamza Shah		
5	Zara Iqbal		

Explanation:

Finds students with payments greater than a specified amount.

Nested Subquery 2

```
SELECT Name
FROM Instructor i
WHERE EXISTS
(
SELECT *
FROM Section s
WHERE s.InstructorID=i.InstructorID
);
```

Results		Messages
	Name	
1	Dr. Hassan	
2	Dr. Ahmed	
3	Dr. Sara	
4	Dr. Bilal	
5	Dr. Ayesha	

Explanation:

Identifies instructors assigned to at least one section.

12. Conclusion

This project successfully demonstrated the practical implementation of a Database Management System using Microsoft SQL Server. The system was designed using EERD concepts, implemented through relational tables, and tested using views, triggers, joins, aggregate functions, and nested queries. The project highlights the importance of database design, integrity constraints, and efficient data retrieval techniques in real-world applications.